

of the user becomes the *value*. The class also offers operations to add and find entries. The code in Java looks like this:

```
package com.glarimy.ums;

public class InMemoryUserRepository {
    private Map<Long, String> entries;

    public InMemoryUserRepository() {
        entries = new HashMap<Long, String>();
    }

    public void add(Long phone, String name) {
        entries.put(phone, name);
    }

    public String find(Long phone) {
        return entries.get(phone);
    }
}
```

Notice that the class does not have the *main()* method. It is just a reusable class that can be used by any other class. The *Application* is one such class that uses *InMemoryUserRepository*.

```
package com.client;

import com.glarimy.ums.InMemoryUserRepository;

public class Application {
    public static void main(String[] args) {
        InMemoryUserRepository repo = new
        InMemoryUserRepository();
        repo.add(9731423166L, "Krishna");
        String name = repo.find(9731423166L);
        System.out.println("Found " + name);
    }
}
```

Note that the *Application* is in the package *com.glarimy.client* and is using an instance of *InMemoryUserRepository*. In other words, the *Application* is consuming the services provided by *InMemoryUserRepository*. Every system is full of such providers and consumers of services.

Service contract

The current implementation of *InMemoryUserRepository* is not persisting the user entries in a database. Instead, it is merely storing them in memory, which makes it a simple cache.

Assume that we are going to sell the UMS to multiple customers. What if some of the customers want database support? There are two options.

The first option is to modify the code of

InMemoryUserRepository to add database support.

However, this won't help us because the other customers do not want this support.

The second option is to develop another class (say *PersistentUserRepository*) with database support. This way, we can build two different jars (one with *InMemoryUserRepository* and the other one with *PersistentUserRepository*) and distribute them as per the customer preference.

In other words, we are planning to have two providers for the same contract — to manage the user data. One provider is with database support and the other is without it.

With two providers at its disposal, the *Application* can choose which one to consume. Conceptually, this looks good! However, the current implementation of the *Application* clearly refers *InMemoryUserRepository* only. Any change of the provider demands a change in the code of the consumer. This is called 'tight coupling'. Does it make sense to couple the providers and consumers tightly? Would you like to make changes to your laptop every time you want to use a different USB device? No, right? Tight coupling is bad!

What makes the laptop and the USB devices couple loosely? The answer lies in the USB interface, which acts as the contract between laptops and various devices.

Let us build such a contract for UMS.

```
package com.glarimy.ums;

public interface UserRepository {
    public void add(Long phone, String name);
    public String find(Long phone);
}
```

The interface *UserRepository* offers a contract with the specifications for the *add()* and *find()* services. It does not offer any implementation. That job is left for the providers of the contract.

The *InMemoryUserRepository* can be made as a provider of this contract by refactoring it as an implementation of the interface.

```
public class InMemoryUserRepository implements UserRepository {
    ...
}
```

As a consumer, the *Application* can now refer to the interface as follows:

```
UserRepository repo = new InMemoryUserRepository();
repo.add(9731423166L, "Krishna");
String name = repo.find(9731423166L);
System.out.println("Found " + name);
```